

Efficiency

KV Cache · GQA · Speculative Decoding · Flash Decoding

AWQ Serving · Chunked Prefill · Throughput vs Latency · Multi-GPU

WEEK 2 · DAY 13

Saturday, July 18 2026

Day 13 of 84

#	Topic	What You Learn
01	The Memory Wall	Why GPU HBM is the bottleneck; FLOPs are not
02	KV Cache Deep Dive	Per-token memory math; GQA vs MHA vs MQA
03	KV Cache Quantization	FP8/INT8 KV cache; 2x concurrency at < 0.3% quality loss
04	Speculative Decoding	Draft + verify; acceptance rate formula; EAGLE-2
05	Flash Decoding	Sequence-parallel KV reads; 8-13x speedup at 16K+
06	Quantization-Aware Serving	AWQ vs GPTQ vs FP8 vs GGUF serving comparison
07	Throughput vs Latency	Chunked prefill; SLO measurement; percentile targets
08	Multi-GPU Efficiency	DP vs TP vs CP; when to use each; load balancing
09	Efficiency Budget Calc	Weights + KV cache <= GPU memory * 0.85
10	FAANG in Production	Meta FP8, Google speculative, Amazon SLO routing
11	Text-to-SQL Thread	Efficient serving of the SQL model; prefix cache hit rate
12	Architect's Lens	6-step efficiency decision framework; impact table

TEXT-TO-SQL CAPSTONE THREAD — Piece 5 preview: efficient serving of the fine-tuned SQL model. AWQ INT4 quantization + prefix caching of system prompt & schema = 95% cache hit rate; speculative decoding with base model as draft (alpha=0.85 on SQL) = 1.9x speedup. Target: 280 tokens/sec on single A100.

Section 1 — The Memory Wall

LLM serving is constrained by GPU HBM (High-Bandwidth Memory), not FLOPS. FLOPS are plentiful on modern GPUs. Memory capacity and bandwidth are not. Every efficiency technique in this lesson attacks the memory wall from a different angle.

LLaMA 3 70B memory requirements (BF16, no optimisation):

```
Model weights:          140 GB   (70B params x 2 bytes/param BF16)
Activations (FW pass):  ~10 GB   (transient, cleared after each step)
KV cache (1 req, 4K ctx): ~1.25 GB per request
KV cache (100 concurrent): ~125 GB
```

```
A100 80GB: 2x A100 needed just for weights (140 GB)
KV budget: 160 GB total - 140 GB weights = 20 GB -> only 16 concurrent requests
```

```
AWQ INT4 quantization: weights -> 35 GB
FP8 KV cache: KV per request -> 0.6 GB
Result: 160 GB - 35 GB = 125 GB for KV -> 208 concurrent requests
```

Efficiency techniques turn 16 concurrent users into 208. Same hardware.

Section 2 — KV Cache: The Math

The KV cache stores Key and Value matrices for every already-processed token in every attention layer. Without it, generating each new token requires $O(n^2)$ recomputation. With it, each decode step is $O(n)$ — read the cache, compute one new K/V, append, attend.

```
# KV cache memory per token (LLaMA 3 8B, GQA, BF16):
num_layers = 32
num_kv_heads = 8      # GQA: 8 KV heads (not 32 like query heads)
head_dim = 128
bytes_bf16 = 2        # BF16 = 2 bytes per float

per_token = 2 * num_layers * num_kv_heads * head_dim * bytes_bf16
# = 2 x 32 x 8 x 128 x 2 = 131,072 bytes = 128 KB per token

# For 4096-token context:
kv_per_req_4k = per_token * 4096 / 1e6  # 512 MB per request

# For LLaMA 3 70B (80 layers, same GQA):
# per_token = 2 x 80 x 8 x 128 x 2 = 327,680 bytes = 320 KB
# kv_per_req_4k = 1.25 GB per request
```

Multi-Head vs Grouped-Query vs Multi-Query Attention

Variant	Q heads	KV heads	KV Cache Size	Quality	Used By
MHA (standard)	32	32	1× baseline	Best	GPT-3, early BERT
GQA	32	8	4× smaller	~0.1% drop	LLaMA 3, Mistral, Gemma

MQA	32	1	32× smaller	~0.5% drop	Falcon, early PaLM
-----	----	---	-------------	------------	--------------------

```
# GQA implementation concept:
# Instead of 32 independent KV heads, 4 query heads SHARE 1 KV head
# At attention time:
#   Q[i] for i in {0,1,2,3} all attend to the SAME K[0], V[0]
#   Q[i] for i in {4,5,6,7} all attend to K[1], V[1]
#   ...

# This means KV cache size = f(num_kv_heads) not f(num_query_heads)
# LLaMA 3 8B: 8 KV heads -> 4x smaller KV than naive MHA
# LLaMA 3 70B: 8 KV heads -> 10x smaller KV than naive MHA (80 query heads)
```

Section 3 — KV Cache Quantization

The KV cache defaults to BF16 (2 bytes per element). Quantizing the KV cache to FP8 or INT8 halves its size, allowing 2× more concurrent requests without extra GPUs.

```
# Quality vs memory trade-off (LLaMA 3 70B, WikiText-2 perplexity):
# BF16 KV:  perplexity = 3.12 (baseline, 0% degradation)
# FP8 KV:   perplexity = 3.13 (+0.03%, negligible)
# INT8 KV:  perplexity = 3.18 (+0.2%, acceptable)
# INT4 KV:  perplexity = 3.35 (+1.5%, noticeable – avoid unless necessary)

# Enable FP8 KV cache in vLLM (H100 required for hardware FP8):
from vllm import LLM

llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    dtype="bfloat16",
    kv_cache_dtype="fp8",           # halves KV memory
    gpu_memory_utilization=0.90,
    max_model_len=8192,
    enable_prefix_caching=True,
)

# A100 fallback: use INT8 KV (software quantization, all GPUs):
# kv_cache_dtype="int8"  – 2x reduction, 0.2% quality cost

# Concurrent request increase (LLaMA 3 8B, single A100 80GB):
# BF16 KV:  ~60 concurrent requests at 4K context
# FP8 KV:   ~120 concurrent requests at 4K context    (2x)
# INT8 KV:  ~120 concurrent requests                  (2x)
```

Section 4 — Speculative Decoding: The Acceptance Rate Formula

Speculative decoding uses a small, fast draft model to propose K tokens speculatively, then verifies all K tokens with the large target model in a single forward pass. Because the target model processes K tokens in parallel rather than one at a time, you get up to $K\times$ latency reduction on the decode phase.

ALGORITHM:

1. Draft model generates K tokens greedily: $[d_1, d_2, d_3, d_4]$
2. Target model processes $[\text{prefix}, d_1, d_2, d_3, d_4]$ in ONE forward pass (outputs logits at each position)
3. Compare draft logits vs target logits at each position:
 - Accept token d_i with probability $\min(1, p_{\text{target}}(d_i) / p_{\text{draft}}(d_i))$
 - If rejected at position j : sample from corrected distribution at j
DISCARD $d_{\{j+1\}} \dots d_K$
4. Guaranteed: output distribution IDENTICAL to target-only decoding (proof: the rejection sampling correction is exact)

KEY PROPERTY: Even when draft is wrong, quality is NEVER degraded.
The only cost of rejection is wasted compute on discarded tokens.

The Acceptance Rate Formula

```
# alpha (alpha) = acceptance rate = P(draft token accepted by target)
# E[tokens accepted per step] = (1 - alpha^(K+1)) / (1 - alpha)
# Speedup = E[tokens per step] / (cost per step)

def expected_tokens(alpha: float, K: int) -> float:
    if alpha == 1.0: return K + 1
    return (1 - alpha**(K+1)) / (1 - alpha)

def speedup(alpha: float, K: int, draft_ratio: float = 0.1) -> float:
    # draft_ratio = draft inference cost / target inference cost
    tokens = expected_tokens(alpha, K)
    cost = 1.0 + K * draft_ratio
    return tokens / cost

# Results table (draft_ratio = 0.1, i.e. 7B draft with 70B target):
# alpha K=2 K=4 K=6 K=8
# 0.50 1.17x 1.25x 1.25x 1.22x <- diminishing returns past K=4
# 0.70 1.44x 1.63x 1.64x 1.58x
# 0.80 1.60x 1.90x 1.96x 1.91x
# 0.85 1.66x 2.03x 2.13x 2.09x
# 0.90 1.72x 2.18x 2.33x 2.32x
# 0.95 1.77x 2.32x 2.54x 2.58x

# Task-specific acceptance rates (LLaMA 3 8B draft, LLaMA 3 70B target):
# SQL generation: alpha = 0.85-0.92 -> K=4 gives 2.0-2.5x speedup
# Code completion: alpha = 0.80-0.92 -> K=4 gives 1.9-2.5x speedup
# Summarization: alpha = 0.75-0.85 -> K=4 gives 1.7-2.0x speedup
# Open-ended chat: alpha = 0.50-0.65 -> K=4 gives 1.2-1.6x speedup
```

EAGLE-2: Best Current Speculative Method

EAGLE-2 (Li et al., 2024) trains a tiny autoregressive head using the target model's internal feature vectors (not just output logits). This gives the draft head much richer context than a separate small model, pushing acceptance rates to 0.85–0.95 across most task types.

```
# vLLM speculative decoding (separate draft model approach):
from vllm import LLM

llm = LLM(
    model="meta-llama/Llama-3.1-70B-Instruct", # target
    speculative_model="meta-llama/Llama-3.1-8B-Instruct", # draft
    num_speculative_tokens=4, # K = 4
    speculative_draft_tensor_parallel_size=1,
    tensor_parallel_size=4, # target on TP=4
    dtype="bfloat16",
    gpu_memory_utilization=0.85,
)

# Equivalent via API server:
# python -m vllm.entrypoints.openai.api_server \
# --model meta-llama/Llama-3.1-70B-Instruct \
# --speculative-model meta-llama/Llama-3.1-8B-Instruct \
# --num-speculative-tokens 4 \
# --tensor-parallel-size 4
```

Section 5 — Flash Decoding

FlashAttention (Day 3) optimises the PREFILL phase. Flash Decoding (Dao et al., Stanford 2023) optimises the DECODE phase for long contexts by parallelising the KV read across the sequence dimension.

```
DECODE PHASE ATTENTION:
  query Q = W_Q x hidden_state[-1]  (just the new token's query)
  keys  K = KV_cache[0..seq_len]     (ALL past keys – grows with sequence)
  values V = KV_cache[0..seq_len]

  attn = softmax(Q x K^T / sqrt(d)) x V

Standard: read ALL of K and V from HBM -> O(seq_len) HBM reads
For 32K tokens: reading K,V from HBM takes ~300ms just for memory ops

FLASH DECODING:
  Split K, V along the SEQUENCE dimension into P chunks
  Each CUDA thread block handles one chunk in parallel
  Each block computes partial attention: partial_attn_i, partial_lse_i
  Final kernel merges via log-sum-exp: log(sum_i exp(partial_lse_i))

Result: P parallel HBM reads of size seq_len/P each
P thread blocks x seq_len/P reads = same total data, but parallelised
Effective bandwidth utilisation: 8-13x higher than standard on long contexts
```

Speedup vs standard attention (A100 80GB, LLaMA 3 8B):

Context	Standard	Flash Decode	Speedup
1K tokens	420 ms	400 ms	1.05x
4K tokens	680 ms	460 ms	1.48x
16K tokens	2,100 ms	450 ms	4.7x
32K tokens	5,800 ms	440 ms	13.2x
128K tokens	OOM	460 ms	infinity (only option)

Flash Decoding is automatic in vLLM, TGI, and FlashAttention v3+
For custom inference loops:

```
from flash_attn.flash_attn_interface import flash_attn_with_kvcache
import math

output = flash_attn_with_kvcache(
    q=query,                # [batch, 1, num_heads, head_dim]
    k_cache=k_cache,        # [batch, max_seq, num_kv_heads, head_dim]
    v_cache=v_cache,        # [batch, max_seq, num_kv_heads, head_dim]
    cache_seqlens=seq_lens,# [batch] – actual lengths per sequence
    softmax_scale=1.0 / math.sqrt(head_dim),
    causal=True,
)
# output: [batch, 1, num_heads, head_dim]
```

Section 6 — Quantization-Aware Serving

Format	VRAM (70B)	Tok/s (bs=32)	Quality drop	Best for
--------	------------	---------------	--------------	----------

BF16	140 GB (OOM 2xA100)	—	0%	Training only
AWQ INT4	35 GB	~180 t/s	-0.5-1%	Latency-sensitive API
GPTQ INT4	35 GB	~160 t/s	-0.5-1.5%	Throughput batch jobs
FP8 (H100)	70 GB	~310 t/s	-0.1-0.3%	H100 production (best)
GGUF Q4_K_M	~4 GB (8B)	~45 t/s (M3)	-1-2%	Local / CPU / edge

AWQ serving with vLLM (most common production choice):

```
from vllm import LLM, SamplingParams
```

```
llm = LLM(
    model="TheBloke/Llama-3-70B-Instruct-AWQ",
    quantization="awq",
    dtype="float16",
    tensor_parallel_size=2,
    gpu_memory_utilization=0.85,
    max_model_len=8192,
    enable_prefix_caching=True,
    kv_cache_dtype="fp8",          # stack: AWQ weights + FP8 KV cache
)
```

GGUF for CPU/Apple Silicon (llama.cpp):

```
from llama_cpp import Llama
```

```
llm_cpu = Llama(
    model_path="./Llama-3-8B.Q4_K_M.gguf",
    n_ctx=4096,
    n_gpu_layers=35,    # offload 35 of 32 layers to Metal GPU
    n_threads=8,
)
```

Apple M3 Max: LLaMA 3 8B Q4_K_M = ~45 tokens/sec

Section 7 — Throughput vs Latency Optimisation

Latency (TTFT + TPOT) and throughput (tokens/sec at scale) pull in opposite directions. Understand which your application needs — then pick the right levers.

LATENCY OPTIMISATIONS (sacrifice some throughput):

- batch_size=1 (lowest P50 latency; wastes GPU)
- Tensor parallel TP=4 (splits each attention op; fast per-request)
- Speculative decoding (K=4; reduces decode latency)
- Prefix caching (eliminates TTFT for repeated system prompts)
- Flash Decoding (critical for 16K+ contexts)

THROUGHPUT OPTIMISATIONS (sacrifice some latency):

- batch_size=64-256 (amortises overhead; max GPU utilisation)
- Continuous batching (fills sequence slots as requests complete)
- Data parallel x N (N copies of model = N× throughput)
- Chunked prefill (interleave prefill with decode; better throughput)

Chunked Prefill

Without chunked prefill, a single long prompt (RAG with 4K context) blocks ALL ongoing decode in the batch until it finishes. Chunked prefill splits that prompt into 512-token chunks and interleaves them with ongoing decode steps.

```
from vllm import LLM

llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    enable_chunked_prefill=True,
    max_num_batched_tokens=2048,  # max total tokens per scheduler step
    max_num_seqs=256,
    gpu_memory_utilization=0.90,
)

# Chunked prefill benchmark (4K-prompt mixed with short prompts, A100):
# Metric                Without          With (chunk=512)  Improvement
# P50 TTFT (4K pmt)    850 ms          320 ms           -62%
# P95 TTFT (4K pmt)    1,400 ms        580 ms           -59%
# P50 TPOT              22 ms          25 ms           +14% (small cost)
# Throughput            baseline        +12%             wins on mixed workloads
```

SLO Benchmarking

```
import time, statistics

def benchmark(llm, prompts, params):
    ttfts, tpots = [], []
    for prompt in prompts:
        t0 = time.perf_counter()
        first, tokens = True, 0
        t1 = None
        for out in llm.generate_streaming(prompt, params):
            if first: t1 = time.perf_counter(); first = False
            tokens += 1
        t2 = time.perf_counter()
        ttfts.append((t1 - t0) * 1e3)  # ms
```



```
        tpots.append((t2 - t1) / max(tokens-1,1) * 1e3)    # ms/tok
def pct(lst, p): return sorted(lst)[int(p * len(lst))]
return {
    "ttft_p50": pct(ttfts, .50), "ttft_p95": pct(ttfts, .95),
    "tpot_p50": pct(tpots, .50), "tpot_p95": pct(tpots, .95),
}

# Production SLOs for a real-time SQL API:
SLOs = {
    "ttft_p50": 200,    # ms — user sees first token in < 200ms
    "ttft_p95": 500,    # ms
    "tpot_p50": 30,     # ms/token — ~33 tokens/sec, smooth streaming
    "tpot_p95": 60,     # ms/token
}
```

Section 8 — Multi-GPU Efficiency: Which Parallelism When

Strategy	When to Use	Latency Impact	Throughput Impact	Requires
Data Parallel (DP)	Model fits on 1 GPU; max throughput	None	N× (N copies)	N× GPUs
Tensor Parallel (TP)	Model too large for 1 GPU	-30-50% TPOT	Limited by TP overhead	NVLink (fast)
Context Parallel (CP)	Very long context (32K+)	Enables long ctx	Scales with length	vLLM 0.6+
Pipeline Parallel (PP)	Very deep models, limited NVLink	PP bubble adds latency	Moderate	P2P bandwidth
TP+DP combined	Production scale serving	Best balance	N× throughput	N× TP GPUs

```
# Data Parallel: 4 model copies on 4 GPUs (LLaMA 3 8B AWQ, each on 1 GPU)
# Simple round-robin load balancer:
import httpx, itertools, asyncio

WORKERS = ["http://localhost:8000", "http://localhost:8001",
            "http://localhost:8002", "http://localhost:8003"]
_pool = itertools.cycle(WORKERS)

async def query(payload: dict) -> dict:
    url = next(_pool)
    async with httpx.AsyncClient(timeout=60) as c:
        r = await c.post(f"{url}/v1/chat/completions", json=payload)
    return r.json()

# TP=2 on 2xA100 for 70B model:
# python -m vllm.entrypoints.openai.api_server \
#   --model meta-llama/Llama-3.1-70B-Instruct \
#   --tensor-parallel-size 2 \
#   --quantization awq \
#   --gpu-memory-utilization 0.85
```

Section 9 — Efficiency Budget Calculator

Before deploying, calculate whether your hardware can serve your target concurrency. The formula: `available_for_kv = total_gpu_mem × 0.85 – weight_gb`. If `available_for_kv / kv_per_request < target_concurrency`, you need more GPUs, quantization, or shorter context.

```
def efficiency_budget(model_b, quant, gpu_type, tp, concurrency, ctx):
    bp = {"bf16":2, "fp8":1, "awq_int4":0.5, "gguf_q4":0.5}
    gm = {"a100_80gb":80, "h100_80gb":80, "l40s_48gb":48}

    weight_gb = model_b * 1e9 * bp[quant] / 1e9
    layers     = 32 if model_b < 15 else 80
```

```

kv_bytes = 2 * layers * 8 * 128 * 2 # GQA (8 kv heads, BF16 KV)
kv_gb    = kv_bytes * ctx / 1e9

total_mem = gm[gpu_type] * tp
avail_kv  = total_mem * 0.85 - weight_gb
max_conc  = int(avail_kv / kv_gb)
ok = max_conc >= concurrency

print(f"Weights: {weight_gb:.1f} GB | KV/req: {kv_gb*1000:.0f} MB | "
      f"Max concurrent: {max_conc} | {'OK' if ok else 'NEED MORE GPUS'}")
return max_conc

# Examples:
efficiency_budget(8, "awq_int4", "a100_80gb", 1, 100, 2048)
# Weights: 4.0 GB | KV/req: 131 MB | Max concurrent: 612 | OK

efficiency_budget(70, "bf16", "a100_80gb", 2, 50, 4096)
# Weights: 140.0 GB | KV/req: 1250 MB | Max concurrent: -4 | NEED MORE GPUS

efficiency_budget(70, "awq_int4", "a100_80gb", 2, 50, 4096)
# Weights: 35.0 GB | KV/req: 1250 MB | Max concurrent: 93 | OK

efficiency_budget(70, "fp8", "h100_80gb", 2, 200, 4096)
# Weights: 70.0 GB | KV/req: 1250 MB | Max concurrent: 90 | NEED MORE GPUS

```

Section 10 — FAANG in Production

Meta — FP8 + Flash Decoding + GQA Stack

Meta's production LLM serving uses three efficiency layers simultaneously: (1) BF16/FP8 inference via NVIDIA Transformer Engine on H100 clusters — FP8 gives 2× throughput over BF16 at 0.1% quality cost. (2) Flash Decoding built into all LLaMA 3 deployments — critical for the 128K-context LLaMA 3.1 variants. (3) GQA in all LLaMA 3 models (8 KV heads, not 32) — 4× smaller KV cache than MHA models. Combined, Meta reports ~1,800 tokens/sec throughput for LLaMA 3 70B at batch=128 on 8× H100 before speculative decoding stacking.

Google — Speculative Decoding at Gemini Scale

Google's Gemini serves billions of requests per day. Gemini Flash uses speculative decoding where Gemini Flash itself serves as a draft model for Gemini Pro (target). The key Google innovation: the draft model shares the target model's embedding layer — reducing draft model memory footprint by ~30% while maintaining acceptance rates at 0.88-0.92 for structured tasks. Google's TPU v5e is optimised for inference bandwidth specifically, giving 2× better tokens/sec per dollar vs A100 for their model sizes.

Amazon — SLO-Aware Request Routing

Amazon Bedrock routes each request to a serving strategy based on declared SLO.
latency_mode='instant': tensor-parallel routing on reserved GPUs, TTFT target < 200ms.
latency_mode='batch': queued for large-batch processing, 10× cheaper but hours of latency.
latency_mode='standard': continuous batching with vLLM-style scheduling. The routing decision happens in < 1ms at the API gateway. This means Amazon serves latency-critical APIs and cost-efficient batch jobs from the same model fleet with each class hitting its SLO.

Netflix — 3x Real-Time Speed Target

Netflix's content description LLM must generate at 3× real-time reading speed (250 WPM human reading → 750 WPM = ~60 tokens/sec target). Achieved via: AWQ INT4 quantization (2× memory on A10G fleet, enabling 2× batch), speculative decoding with a 3B draft (1.7× speedup on structured description text, alpha ≈ 0.83), and prefix caching of the identical system prompt across all 260M users (shared prefix = 500 tokens; 95% cache hit rate → saves 200-400ms TTFT). Combined baseline → achieved: 22 tokens/sec → 68 tokens/sec.

Microsoft — ZeRO-Inference and Efficiency Tier Pricing

Microsoft's DeepSpeed ZeRO-Inference offloads KV cache to CPU/NVMe when GPU is insufficient — enabling 70B inference on a single A100 by streaming layers from CPU RAM (at ~4 tokens/sec; not for latency-sensitive production, but useful for offline batch jobs where cost matters more than speed). Azure AI Foundry exposes efficiency as a first-class API param: `compute_tier='standard'` (BF16 full quality), `'optimized'` (AWQ+speculative, 2× throughput), `'economy'` (aggressive quant, 4× throughput). Each tier has different \$/1M token pricing, making efficiency a product feature rather than an infra decision.

Section 11 — Text-to-SQL Thread: Efficient SQL Model Serving

The fine-tuned SQL model has structural advantages for efficiency. SQL generation is templated, so speculative decoding acceptance rates are high. The system prompt + schema is shared across ALL requests, so prefix caching eliminates most of the TTFT cost. This section shows the full efficient setup.

```
#!/usr/bin/env python3
"""Efficient Text-to-SQL serving – Capstone Piece 5 preview."""
from vllm import LLM, SamplingParams
import time

SCHEMA = """
CREATE TABLE users (user_id INT, email TEXT, country TEXT, created_at DATE);
CREATE TABLE orders (order_id INT, user_id INT, amount FLOAT, status TEXT);
CREATE TABLE products (product_id INT, name TEXT, category TEXT, price FLOAT);
CREATE TABLE order_items (order_id INT, product_id INT, quantity INT, unit_price FLOAT);
"""

SYSTEM = f"You are a SQL expert. Schema:\n{SCHEMA}\nOutput SQL only."

llm = LLM(
    model="/models/sql-llama-8b-awq",
    quantization="awq",
    dtype="float16",
    tensor_parallel_size=1,
    gpu_memory_utilization=0.90,
    max_model_len=4096,
    enable_prefix_caching=True,      # caches SYSTEM + SCHEMA (99% hit rate)
    # speculative_model="meta-llama/Llama-3.1-8B", # draft for spec decoding
    # num_speculative_tokens=4,
)

sql_params = SamplingParams(
    temperature=0.05,
    max_tokens=512,
    stop=["``", "\n\n\n"],
)

def text_to_sql_batch(questions: list[str]) -> list[str]:
    prompts = [
        f"<|system|>\n{SYSTEM}\n<|user|>\n{q}\n<|assistant|>"
        for q in questions
    ]
    t0 = time.perf_counter()
    outputs = llm.generate(prompts, sql_params)
    elapsed = time.perf_counter() - t0
    total_tokens = sum(len(o.outputs[0].token_ids) for o in outputs)
    print(f"{len(questions)} reqs | {total_tokens} tokens | "
          f"{elapsed:.2f}s | {total_tokens/elapsed:.0f} tok/s")
    return [o.outputs[0].text.strip() for o in outputs]

# Efficiency profile (fine-tuned 8B AWQ, single A100 80GB):
# Single request (prefix cache HIT): 55-70 tok/s, TTFT < 50ms
```

```
# Single request (prefix cache MISS): 55-70 tok/s, TTFT ~ 300ms
# Batch of 32 (cache HIT): 280-320 tok/s
# Prefix cache hit rate for SQL: ~95% (same schema every request)
# Speculative decoding speedup: ~1.9x (alpha=0.85 for SQL templates)
```

Section 12 — Architect's Lens: Efficiency Decision Framework

6-STEP EFFICIENCY DECISION FRAMEWORK (apply in order):

- STEP 1: RIGHT-SIZE THE MODEL
- Fine-tuned 8B vs 70B on YOUR test set. Within 5%? Use 8B.
 - 8B AWQ INT4: 4 GB, 280 tok/s per A100. 70B AWQ INT4: 35 GB, 180 tok/s per 2xA100.
 - Correct model choice saves more than any other technique.
- STEP 2: QUANTIZE
- AWQ INT4: 2-4x memory reduction, <1% quality loss. One-time cost.
 - FP8 on H100: best ratio; 0.1% quality cost. Use if you have H100s.
 - Stack: AWQ weights + FP8 KV cache = 4x memory from two techniques.
- STEP 3: SPECULATIVE DECODING (structured output only)
- SQL, code, JSON, templates: alpha >= 0.8. K=4 gives 1.9-2.5x TPOT reduction.
 - Open-ended chat: alpha ~ 0.6. K=4 gives only 1.4x. Not worth draft overhead.
 - Enable when: latency is the constraint AND task is structured.
- STEP 4: PREFIX CACHING (shared prefixes)
- If >30% of requests share system prompt or schema, prefix caching is free.
 - SQL model: 99% cache hit rate -> saves 200-400ms TTFT on every request.
 - Enable unconditionally for any API with a fixed system prompt.
- STEP 5: KV CACHE QUANTIZATION (high concurrency)
- >50 concurrent users? Enable FP8 KV cache (2x concurrency for 0.3% quality).
 - Run `efficiency_budget()` first to know if you need it.
- STEP 6: CHUNKED PREFILL (mixed short+long prompts)
- Mixed workload with RAG (4K+ prompts) AND short queries?
 - Enable chunked prefill. Reduces p95 TTFT by 50-60% with <15% TPOT cost.

Technique	Latency	Throughput	Quality	Complexity
AWQ INT4	-5% TTFT	+2x	-1%	Low (run once)
FP8 KV cache	same	+2x conc.	-0.3%	Low (flag)
Spec decoding K=4	-40% TPOT	same	0%	Medium (draft)
Prefix caching	-60% TTFT*	+20-40%	0%	Low (flag)
Chunked prefill	-30% p95 TTFT	+10%	0%	Low (flag)
Data parallel x2	same	+2x	0%	Medium (2 GPUs)
Flash Decoding	-50% TPOT†	+20%	0%	Low (built-in)

* when prefix cache hits | † long context (16K+) only

Section 13 — Hands-On Exercises

Exercise 1: KV Cache Budget

```
# Which of these setups can handle 200 concurrent users at 4K context?
efficiency_budget(8, "bf16", "a100_80gb", 1, 200, 4096) # A
efficiency_budget(8, "awq_int4", "a100_80gb", 1, 200, 4096) # B
efficiency_budget(70, "awq_int4", "a100_80gb", 2, 200, 4096) # C
efficiency_budget(70, "fp8", "h100_80gb", 4, 200, 4096) # D
# Answer: B and D pass; A fails (bf16 too heavy); C marginal at 4K ctx
```

Exercise 2: Find the Optimal K for Your Task

```
# SQL generation model, alpha=0.87 (measured), draft is 10% cost of target
for K in range(1, 9):
    tokens = expected_tokens(0.87, K)
    sp = speedup(0.87, K, draft_ratio=0.1)
    print(f"K={K}: {tokens:.2f} tokens/step, {sp:.2f}x speedup")
# Find the K where additional gain < 5% (usually K=4 for alpha=0.87)
```

Exercise 3: Enable All Efficiency Flags

```
# Baseline (no optimisation):
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3.1-8B-Instruct \
    --dtype bfloat16 --port 8000

# Optimised (all flags):
python -m vllm.entrypoints.openai.api_server \
    --model TheBloke/Llama-3-8B-Instruct-AWQ \
    --quantization awq --dtype float16 \
    --kv-cache-dtype fp8 \
    --enable-prefix-caching \
    --enable-chunked-prefill \
    --max-num-batched-tokens 2048 \
    --gpu-memory-utilization 0.90 \
    --port 8001

# Run benchmark (vLLM built-in):
python -m vllm.benchmark_serving \
    --base-url http://localhost:8001 \
    --model TheBloke/Llama-3-8B-Instruct-AWQ \
    --dataset-name sharegpt \
    --num-prompts 1000 --request-rate 20
```

Summary — Day 13 Key Concepts

Concept	One-Line Definition
KV cache	Stores K,V matrices for all past tokens; avoids $O(n^2)$ recomputation
GQA	4-8 KV heads shared by 32 Q heads; 4-8x smaller KV than MHA
KV quantization	FP8/INT8 KV; 2x more concurrent requests at <0.3% quality cost
Speculative decoding	Draft K tokens; verify in 1 target pass; $E[\text{tokens}] = (1 - a^{(K+1)}) / (1 - a)$
Acceptance rate (a)	$P(\text{draft token matches target})$; 0.85+ for structured, 0.6 for chat
EAGLE-2	Draft head on target features; best method; $a = 0.85 - 0.95$
Flash Decoding	Parallel KV reads along seq dim; 8-13x speedup at 16K+ context
Chunked prefill	Split long prompts into chunks; interleave with decode; -50% p95 TTFT
AWQ INT4	Activation-aware quantization; 4x memory; <1% quality loss
GGUF / llama.cpp	CPU/edge format; Q4_K_M = best 4-bit quality; runs 8B on MacBook
DP vs TP	DP = copies of whole model; TP = splits one model across GPUs
Efficiency budget	$\text{available_for_kv} = \text{GPU_mem} * 0.85 - \text{weights}$; drives max concurrency

TOMORROW — Day 14: Week 2 Project

Build the complete serving stack for the Text-to-SQL model: AWQ quantization of the Day 9 fine-tuned model, vLLM deployment with all efficiency flags enabled, integration with the Day 12 evaluation harness, latency benchmarking against SLOs, and a production-ready FastAPI endpoint with health checks and Prometheus metrics.

Eagle Eye Addendum — Day 13: PagedAttention (vLLM)

PagedAttention is vLLM's memory management technique for the KV cache. It divides the KV cache into non-contiguous fixed-size pages (blocks), eliminating the need to pre-allocate a contiguous memory region per sequence. This is the key innovation that enables continuous batching at scale.

```
# PagedAttention – memory management for KV cache

# PROBLEM (naive serving):
# KV cache must be pre-allocated contiguously per request.
# A 2048-token KV cache for LLaMA-3-8B ≈ 1 GB contiguous GPU RAM.
# With 16 concurrent requests: 16 GB – but fragmentation wastes 30-40%.

# PAGED ATTENTION SOLUTION:
# KV cache is split into pages (blocks), each holding K,V for 16 tokens.
# Pages allocated on demand from a global page pool.
# A single request maps to a linked list of pages – not contiguous.

# Result in vLLM:
# - Near-zero KV cache waste (< 4% fragmentation vs 30-40% before)
# - 3x higher throughput than HuggingFace text-generation-inference
# - Enables prefix sharing: multiple requests sharing a system prompt
#   share the SAME physical KV cache pages (copy-on-write semantics)

# Example: 1000-token shared system prompt across 100 concurrent users
# Naive:      100 × 1000 × KV_per_token = 100x memory
# PagedAttention: 1 × 1000 × KV_per_token = 1x memory (shared pages)

# PagedAttention was the core innovation in the vLLM paper (2023),
# enabling continuous batching + high GPU utilization simultaneously.
```